TH-1 Bus Replay Protocol Specification

The TH-1 bus replay protocol connects a bus capture and replay device to a host computer. The device attaches to a bus under test, such as I^2C, samples its lines and streams every level change to the host, where the traffic is decoded, inspected and replayed. This document specifies how captures are encoded and streamed, how the host configures the device, and how transports carry both.

Status: draft

Version	**Date**	**Changes**
1.0	2026-10-19	First version.

Contents

1 Scope **3**

2 Terminology **4**

3 Design principles **4**

4 Channel model **5**

4.1 Control channel 5

4.2 Data channels 5

4.3 Mapping 5

4.4 Contract 6

5 Time and records **6**

5.1 Tick 6

5.2 Record 6

5.3 Gap 7

6 Heartbeat **7**

6.1 Content 7

6.2 Emission 7

6.3 Interval 8

6.4 Pending heartbeat 8

6.5 Position 8

6.6 Host use 8

7 Loss **8**

7.1 Device behaviour 9

7.2 Loss report 9

7.3 Host behaviour 9

7.4 Transport loss 10

8 Control messages **10**

8.1 Hello 10

8.2 Registers 11

8.3 Configuration 12

8.4 Loss report 13

8.5 Capture lifecycle 13

8.6 Replay 14

9 Transports **14**

9.1 Data bindings 14

9.1.1 Serial (UART) 14

9.2 Control bindings 14

9.2.1 SPI 14

10 Open items **15**

1 Scope

In scope:

- The channel model, and how logical channels map onto data channels.
- The time model, records and heartbeats on the device → host data path.
- Loss detection and reporting.
- Control messages: hello, registers, configuration, loss report, capture lifecycle.
- Transports: the contract every transport must meet, and the bindings this document defines.

Out of scope:

- The device's internal design (sampling method, buffer implementation).
- The host interface and its data model.
- Protocol decoding (I^2C or otherwise). Decode is a host-side step over the raw pin states this protocol delivers.

2 Terminology

Term	Meaning
Device	The capture/replay hardware attached to the bus under test.
Host	The PC running the analyser interface.
Logical channel	One device pin, with a channel number fixed by the device, enabled in the configuration. The unit the protocol reasons about.
Data channel	A transport-provided byte stream that carries pin states.
Control channel	The single transport-provided message channel for everything that is not pin state.
Tick	One sample period. Global, shared by all logical channels.
Record	A level change on one logical channel (§5.2).
Heartbeat	A periodic entry, not a record, that marks the same instant on every logical channel (§6).
Gap	The number of ticks between a record and its reference point (§5.3).
Register	A device setting with an ID, written by the host (§8.2).
Grid point	A tick at which a heartbeat is due: every C ticks from the start of the capture (§6.2).
Position	A tick on the capture timeline, as grid count and offset (§6.5).
Entry	A record or a heartbeat on a data channel.
Binding	The part of a transport that carries one data channel, or the control channel, over one medium (§9).
Untrusted span	A span of a data channel the host cannot trust after transport loss (§7.4).

Symbol	Meaning
N	The number of channels the device has (§8.1).
n	The number of enabled logical channels (§8.2).
D	The number of data channels (§4).
C	The heartbeat interval in ticks (§6.3).
W	The width of a record in bytes, per data channel (§5.2).
t	The width of a record's tag in bits, per data channel (§5.2).

3 Design principles

1. **Streamed.** Captures flow to the host while the capture runs, not as a buffer download after it stops.
2. **Lightweight.** Minimal bytes on the wire and minimal device-side work. The host may spend processing freely.
3. **Separate control channel.** Everything that is not pin state goes on it.
4. **Data channels carry only pin states**, in either direction.
5. **Transport agnostic.** The core works over any transport that meets the contract (§4.4).
6. **Bus-protocol agnostic.** Nothing in the protocol may assume the channel count, clock rate

or traffic density of a particular bus protocol.

7. **Fixed sample rate**, set in the configuration and constant for a capture.
8. **Encoded.** Captures are per-channel change records, not raw fixed-rate samples.
9. **Buffered.** The device has one buffer and one tick counter. Capture may outrun the transport; the backlog drains while the bus is idle.

4 Channel model

The protocol defines **one control channel and D data channels**. Every transport must provide both. How it does so, and the value of D, belong to the transport (§9).

4.1 Control channel

- Two-way, low rate.
- Message-framed and integrity-checked by its control binding (§9.2): a message arrives whole and intact, or not at all.
- Delivery is not guaranteed.
- Device → host: hello (§8.1), configuration replies (§8.3), loss reports (§7.2).
- Host → device: configuration (§8.3).
- Messages: §8.

4.2 Data channels

- Each data channel is an ordered byte stream in each direction.
- They carry pin states and nothing else.
  - Device → host: records (§5) and heartbeats (§6).
  - Host → device: replay (§8.6).
- The core protocol adds no escapes or framing to a data channel: entries follow one another directly. Any framing a transport needs belongs to that transport.
- D is set by the transport and must not limit how many logical channels exist.
- The protocol sets no maximum number of logical channels. The limit is the device's.
- Which logical channels each data channel carries is set by the mapping (§4.3).

4.3 Mapping

This defines the device → host direction. The host → device direction is **TBD** (§8.6).

- The transport numbers its data channels $0 \dots D-1$. The hello carries D (§8.1).
- The logical channels, in ascending channel number, are split into D contiguous blocks, one per data channel, in data channel order. A record carries its index within the block (§5.2).
- The split register (§8.2) sets where the blocks split. Empty, its default, means automatic: the blocks are $\lfloor n/D \rfloor$ channels, and the first $n \bmod D$ blocks have one more.
- Otherwise, a channel in the split set starts a new block. It need not be enabled: the split is by channel number.
- A block may be empty, for example with fewer than $D-1$ channels in the split set, or none enabled between two of them. Its data channel then carries no records, only heartbeats (§6.2).

For example, with channels 0, 1, 2, 3, 4, 6 and 7 enabled and $D = 3$:

Split	Data channel 0	Data channel 1	Data channel 2
empty	0, 1, 2	3, 4	6, 7
{2, 6}	0, 1	2, 3, 4	6, 7
{6}	0, 1, 2, 3, 4	6, 7	none

4.4 Contract

Across its data channels, a transport must guarantee:

- On each data channel, records and heartbeats arrive in the order the device wrote them.
- Every heartbeat arrives intact on every device → host data channel (§6.2), or the transport detects that it did not (§7.4).

Records carry no integrity guarantee.

Ordering *between* logical channels carried on different data channels is not guaranteed. The host aligns them using heartbeats (§6.6).

A transport must also give the host a way to reset the device (§8.5).

5 Time and records

5.1 Tick

One tick is one sample period. The sample rate is fixed by the configuration for the whole capture. There is one tick counter on the device, shared by all logical channels.

The device samples every tick. It does not start a capture at a sample rate it cannot sustain (§8.3).

5.2 Record

A record is emitted for every sampled level change on a logical channel. Nothing is filtered. Logically it is:

```
{ channel, new level, gap }
```

On the wire it is one value of W bytes, low byte first:

- $v = +\text{gap}$ for new level 1, $v = -\text{gap}$ for new level 0.
- $z = (v \ll 1) \oplus (v \gg 31)$, with an arithmetic shift (ZigZag). Bit 0 of z is 1 for level 0, and 0 for level 1.
- **Tag.** The record's channel is identified by its index within the data channel's block (§4.3), counted from 0 in ascending channel number, in $t = \lceil \log_2 b \rceil$ bits, where b is the block size. A data channel carrying one logical channel or none has $t = 0$: no tag.
- The value is $(z \ll t) \mid \text{index}$.
- W is the smallest number of bytes that holds $(2C + 1) \times 2^t - 1$. Every record on a data channel is W bytes.

W and t are per data channel. Both sides derive them from the configuration.

5.3 Gap

A record's gap is the number of ticks since the more recent of:

- the previous record on the same logical channel, or
- the most recent heartbeat written before it (§6).

A heartbeat resets the gap reference of every logical channel at once. A gap is at least 1.

A record that was dropped (§7.1) was never written and resets nothing.

A gap never exceeds C (§6.4).

6 Heartbeat

A periodic marker of the same instant on every logical channel. It re-anchors the timeline so that alignment between channels and recovery from errors do not depend on summed gaps alone.

6.1 Content

- A heartbeat is one entry for all logical channels, not a record. It is marked by a value of 0. No record has that value, since a record's z is at least 1 (§5.2, §5.3).
- It carries the level of every logical channel as a bitmask, one bit per logical channel. A heartbeat is therefore a keyframe for all channels in one write.
- The bitmask holds each channel's level at the end of the heartbeat's tick, including any change on that tick.
- It carries a grid index: the number of its grid point, counted from the start of the capture, mod 256.

On the wire:

Field	**Size**	**Contents**
Marker	W bytes (§5.2)	all zero, with no tag
Grid index	1 byte	grid index
Bitmask	$\lceil n/8 \rceil$ bytes	the i-th logical channel in ascending channel number, counted from 0, is bit (i mod 8) of byte $\lfloor i/8 \rfloor$. Unused bits are 0.

A heartbeat is $W + 1 + \lceil n/8 \rceil$ bytes, with the W of the data channel it is on. It is fixed for the capture.

6.2 Emission

- A capture begins with a heartbeat.
- A heartbeat marks tick 0 at its grid point, even if written later (§6.4), and resets every gap reference (§5.3).
- A heartbeat is due at every grid point.
- Within a tick, records are written before the heartbeat.
- Heartbeats are global: the device writes one for all logical channels. It goes on every device $\rightarrow$ host data channel, with that data channel's W, so each stream can anchor itself.

6.3 Interval

C is set by the configuration (§8.2), within the range the device lists in its hello (§8.1). The protocol fixes no default or range.

6.4 Pending heartbeat

A heartbeat is never skipped.

- If the buffer has no room for a heartbeat at its grid point, it is pending. The device keeps its grid index and the levels at the grid point.
- A pending heartbeat is written as soon as there is room, before any record. Records that arrive meanwhile and do not fit after it are dropped (§7.1). Records are never held back.
- The device does not start a capture with a C shorter than the time needed to send one heartbeat on every data channel (§8.3). While a heartbeat is pending nothing else is written and the buffer drains, so it is always written before the next grid point.
- A transport that stops draining is outside the protocol.

Every record therefore follows the heartbeat of the grid point before it, and a gap never exceeds C.

6.5 Position

A position is a tick on the capture timeline, sent where a message needs to name one (§8.4).

Field	Size	Contents
Grid count	4 bytes	the number of the grid point at or before the tick, counted from the start of the capture
Offset	4 bytes	ticks since that grid point, $0 \dots C - 1$

- The tick is grid count $\times C +$ offset.
- The grid count is mod 2^{32}. The host unwraps it against the grid points it has seen.
- A heartbeat's grid index is its grid count mod 256 (§6.1).
- A position names a tick, not an epoch. A tick that is a grid point always has offset 0, even for a record on that tick whose gap counts from the previous heartbeat (§6.2).

6.6 Host use

- **Alignment.** Heartbeats with the same grid index mark the same instant on every data channel. The host aligns data channels on that.
- **Known extent.** A logical channel's timeline is known up to its latest record or heartbeat. A quiet channel is therefore known to within one interval C.
- **Epoch length.** The time between two heartbeats received on a data channel is their grid index difference (mod 256) $\times C$ ticks. This holds across heartbeats lost in transport, and is exact unless 256 or more intervals pass between two received heartbeats.

7 Loss

Loss is never repaired. There are two kinds:

- **Device loss.** The buffer fills when capture outruns the transport. The device detects the loss itself and reports it on the control channel (§7.1–§7.3).
- **Transport loss.** Bytes lost or corrupted on a data channel between device and host (§7.4).

7.1 Device behaviour

- A record is written only if the buffer has room for it (W bytes, §5.2). Otherwise it is dropped whole, never partly written, and is counted (§7.2).
- While a heartbeat is pending, a record is written only after it, and only if both fit (§6.4).
- Nothing is sent on the data path. Gaps after a drop stay correct (§5.3).

7.2 Loss report

A report says which channels lost records and how many (the counters), and when (the loss start and end).

The device keeps a **lost-records** counter for each logical channel: the number of records on that channel dropped for lack of buffer room. Each counter:

- is an unsigned 32-bit integer,
- only increases, and wraps at 2^{32},
- is never cleared except by a device reset (§8.5).

Drops are grouped into losses, and the device reports each loss on the control channel (§8.4):

- **Start.** A loss starts when a record is dropped while no loss is open. The position (§6.5) of that drop is the loss start.
- **End.** The position of the latest drop in the loss is the loss end. The loss is over when the next entry, a record or a heartbeat, is written. Heartbeats are never skipped (§6.4), so a loss is over within one interval of the buffer having room, even on an idle bus.
- **Report.** When a loss is over, a report is due. It carries the loss start, the loss end and every counter's value at that point. The device sends it when the control channel can take it.
- **Merging.** If the previous report has not been sent when a loss is over, the loss is merged into it: the report keeps its loss start and takes the new loss end and counters. At most one report waits at a time.
- **Sequence.** Each report carries a sequence number. It is 0 for the first report after a boot and goes up by one for each report sent.

7.3 Host behaviour

- **Loss report.** Report an overrun. For each logical channel whose counter rose, mark possible lost transitions from the loss start to the loss end. Records inside that range are kept. Other channels are not marked. Do not repair or guess lost transitions.
- **Counters.** The host keeps the last value of each counter and takes the unsigned difference, so a wrap reads correctly. After a reset it starts again from 0.
- **Sequence skip.** A report was lost. The counter differences are still exact. For each channel whose counter rose, mark from the loss end of the previous report received to the loss end of this one, or from the start of the capture if none was received since the reset.
- **Merged range.** A merged report's range can include records that were written. It may mark more than was lost, never less.
- **Repeated level.** After a dropped record, the next record on that channel may carry the

channel's previous level. The host accepts it.

7.4 Transport loss

The transport must detect loss or corruption of heartbeats (§4.4). It may also detect it for records.

- When the transport detects loss or corruption on a data channel, the host marks an **untrusted span** on every logical channel carried on that data channel, from the last point the transport confirmed good to the point the loss was detected.
- Levels and gaps are trusted again from the next heartbeat the transport confirms good.
- Heartbeats lost inside the span are accounted for by the grid index (§6.6).

8 Control messages

Every control message has the same shape. The transport delivers it whole (§4.1), so its length is known.

Field	Size	Contents
Type	1 byte	which message
Body	rest of the message	depends on the type

Type	Direction	Message
`0x01`	device → host	hello (§8.1)
`0x02`	host → device	configuration (§8.3)
`0x03`	device → host	configuration reply (§8.3)
`0x04`	device → host	loss report (§8.4)

- Integers are unsigned and sent low byte first.
- **str**: a 1-byte length, then that many ASCII bytes, with no terminator. It may be empty.
- **bytes**: a 1-byte length, then that many raw bytes.
- A receiver ignores a message of a type it does not know.

8.1 Hello

Sent by the device on every boot (power-on or reset), device → host. It tells the host the device is ready.

Field	Size	Contents
Type	1 byte	`0x01`
Version major	1 byte	major part of this document's version (change history above). The host refuses one it does not know.
Version minor	1 byte	minor part of this document's version. The host accepts a newer one.
Model	str	device model
Firmware	str	firmware version
Unique ID	bytes	an ID unique to this unit
Buffer size	4 bytes	capture buffer size in bytes
Channel count	2 bytes	N
Data channel count	1 byte	D (§4.3)
Channel names	$N\times$ str	one per channel, in channel number order
Register count	1 byte	number of register entries
Registers	per entry	one entry per register (§8.2), below

Register entry:

Field	Size	Contents
ID	1 byte	register ID
Type	1 byte	type code
Flags	1 byte	bit 0: writable. Other bits are 0.
Name	str	short name
Unit	str	unit; may be empty
Allowed values	kind and data	the register's allowed values (§8.2)
Current value	per the type	after a boot, the default

There is no query message. To read the device's state, the host resets it and reads the hello.

8.2 Registers

Every setting is a register with a 1-byte ID. IDs `0x00`–`0x7F` are reserved for the core, which defines the registers below with fixed IDs, the same on every device. IDs `0x80`–`0xFF` are for registers a device or transport adds; the host learns them from the hello.

ID	Register	Type	Unit	Default	Allowed values
`0x00`	capture	bool	none	off	list: on. Only while off (§8.5)
`0x01`	sample rate	u32	Hz	device	device, as a range, list or divider
`0x02`	C	u32	ticks	device	device, as a range, minimum at least 1
`0x03`	enabled channels	channel set	none	device	any
`0x04`	split	channel set	none	empty	any

Code	Type	Size	Encoding
`0x00`	bool	1 byte	`00` off, `01` on
`0x01`	u32	4 bytes	unsigned, low byte first
`0x02`	channel set	$\lceil N/8 \rceil$ bytes	Channel i is bit (i mod 8) of byte $\lfloor i/8 \rfloor$; 1 means in the set. Unused bits are 0.

Other type codes are reserved.

Allowed values, as the hello lists them (§8.1), start with a kind byte. Values are in the register's type.

Kind	Meaning	Followed by
`0x00`	any value of the type	nothing
`0x01`	range, inclusive	min, max
`0x02`	list	count (u16), then the values
`0x03`	divider	base, min divisor, max divisor (each u32)

- **Divider.** The allowed values are base $\div d$ for every whole d from min to max divisor for which the result is whole. The min divisor is at least 1. For u32 registers only.
- **Sample rate.** The device accepts only rates it can produce exactly; it does not round.
- **Channel sets.** A bit for a channel the device does not have is rejected.
- **Enabled channels.** At least one channel must be enabled to start a capture (§8.3).

8.3 Configuration

The host writes one register per configuration message, and sends the next only after the reply to the last.

Field	Size	Contents
Type	1 byte	`0x02`
Register ID	1 byte	the register (§8.2)
Value	per the register's type	the new value

The device replies accepted, or rejected with a reason:

Field	Size	Contents
Type	1 byte	`0x03`
Register ID	1 byte	the register written
Status	1 byte	`0x00` accepted, otherwise a reason code
Message	str	text from the device; may be empty
Value	per the type, rest of the message	the register's value after the write: the new value if accepted, the unchanged value if rejected, empty if the register is unknown

Code	Reason
`0x01`	unknown register
`0x02`	register not writable
`0x03`	wrong value length for the type, or message too short
`0x04`	value not in the allowed values
`0x05`	capture already started
`0x06`	start: sample rate not sustainable
`0x07`	start: C too short to send one heartbeat on every data channel
`0x08`	start: no channel enabled
`0x09`	start: split set holds more than $D - 1$ channels
`0x80`–`0xFF`	device-specific; the message explains it

A start rejection (`0x06`–`0x09`) is a reply to the capture register, `0x00`.

- Each write is checked on its own: the register exists and is writable, the value has the right type and is within its allowed values, and capture has not started. If a check fails, the write is rejected and nothing changes.
- If no reply arrives (§4.1), the host may send the same write again. Every write can be repeated safely: a repeated start after one that succeeded is answered `0x05`, which means capture is on.
- Combined limits are checked when capture is started: the sample rate must be one the device can sustain (§5.1), C must be long enough to send one heartbeat on every data channel (§6.4), at least one channel must be enabled, and the split set may hold at most $D - 1$ channels (§4.3). If they fail, the start is rejected and capture stays off. Before start, registers may be written in any order.

8.4 Loss report

Sent by the device after a loss (§7.2).

Field	Size	Contents
Type	1 byte	`0x04`
Sequence	4 bytes	report sequence number
Loss start	8 bytes	position (§6.5) of the first drop
Loss end	8 bytes	position of the latest drop
Lost records	$n \times 4$ bytes	one counter per logical channel, in ascending channel number

8.5 Capture lifecycle

- There are no triggers.
- Capture starts when the host sets the capture register (§8.2) to on. This is one-way: it cannot be set back to off.
- On an accepted start, the device sends the reply before it writes the first heartbeat.
- Once capture has started, the device rejects every configuration message.
- Capture stops when the host resets the device. This is outside the protocol: buffered data is discarded, and the device returns to its power-on state with capture off and must be configured again.

8.6 Replay

Replay travels host → device on the data channels. Its format is **TBD**.

9 Transports

A transport is made of **bindings**. A data binding carries one data channel over one medium. A control binding carries the control channel over one medium. A device combines one control binding with D data channels, each on a data binding, and a way to reset the device (§4.4).

- Bindings combine freely. The combination is not announced: the host knows it before the hello.
- A binding setting the host must change is a device register (`0x80`–`0xFF`, §8.2).
- Everything else (pins, rates, modes, endpoints) is implementation.

9.1 Data bindings

A data binding defines:

- how the data channel's bytes are carried, in order (§4.4);
- how loss or corruption of a heartbeat is detected (§4.4, §7.4).

9.1.1 Serial (UART)

- One UART is one data channel. The device's TX carries device → host; its RX carries host → device (§8.6).
- A **check byte** follows every heartbeat on device → host. It is a CRC-8 over every byte sent since the previous check byte, up to and including this heartbeat; for the first heartbeat, from the start of the capture.
- CRC-8: polynomial `0x07`, initial value `0xFF`, MSB first, no reflection, no final XOR. Reset after each check byte.
- A mismatch marks an untrusted span (§7.4) from the previous good check byte to this one.
- The stream has no other resynchronisation aid. After a mismatch the host finds the next heartbeat itself; how is host implementation.
- A heartbeat on serial is $W + 1 + \lceil n/8 \rceil + 1$ bytes, counting its check byte. The start check on C (§6.4) counts it too.

9.2 Control bindings

A control binding defines:

- how a message arrives whole, with its length known (§4.1, §8);
- how integrity is checked (§4.1);
- how the device sends a message unprompted (hello, loss report);
- the maximum message length.

9.2.1 SPI

- SCLK, MOSI, MISO and CS are normal SPI: the host is master, the device is slave.

- **IRQ** is driven by the device, active low, and pulled up by the host, so an unpowered or reset device reads as nothing waiting. The device asserts it while it has a message to send, and the host then runs a transaction.
- The host may start a transaction at any time to send its own message. It does not poll.

A transaction runs from CS low to CS high. Both lines carry the same layout at once: MOSI the host's message, MISO the device's.

Field	Size	Contents
Length	2 bytes	length L of this side's message, low byte first; 0 for no message
Message	L bytes	the control message (§8)
Check	2 bytes	CRC-16 over Length and Message; absent when $L = 0$

- The host reads the device's Length first and clocks until both sides' messages and checks are through, padding its own side with `00`.
- A device Length of `0xFFFF` means the device was not ready. Nothing moved in either direction, and the host retries.
- One message per side per transaction. IRQ stays asserted while more are queued.
- The maximum message length is `0xFFFE` bytes.
- CRC-16: polynomial `0x1021`, initial value `0xFFFF` (CRC-16/CCITT-FALSE). A receiver discards a message whose check fails.
- A message is sent once its transaction ends. There are no acknowledgements (§4.1).

10 Open items

Replay

#	Item	Section
8	Replay	§8.6

Transports

#	Item	Section
11	USB data and control bindings	§9
12	Serial control binding	§9